

**ARCHLENS: REACT NATİVE PROJELERİ İÇİN STATİK MİMARİ
ANALİZ SİSTEMİ**

Lisans Tezi

Sidi Mohamed BABA

SAMSUN ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

Samsun, Haziran 2026



SAMSUN ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

ARCHLENS: REACT NATİVE PROJELERİ İÇİN STATİK
MİMARİ ANALİZ SİSTEMİ

Sidi Mohamed BABA

9201118073

LİSANS TEZİ

Yazılım Mühendisliği Bölümü

Danışman: Doç. Dr. Nurettin ŞENYER

08 Haziran 2026

BİTİRME PROJESİ ONAYI

Sidi Mohamed BABA tarafından hazırlanan "ArchLens: React Native Projeleri için Statik Mimari Analiz Sistemi" lisans tezi 10/06/2026 tarihinde aşağıdaki jüri tarafından değerlendirilerek Samsun Üniversitesi Mühendislik Fakültesi Eğitim-Öğretim ve Sınav Yönetmeliği'nin ilgili maddeleri uyarınca Yazılım Mühendisliği Bölümünde Lisans tezi olarak kabul edilmiştir.

Jüri Üyeleri

Üye (Danışman) : Doç. Dr. Nurettin ŞENYER

İmza :

Üye : Dr. Öğr. Üyesi ÖZGÜR TONKAL

İmza :

Üye : Doç. Dr. SELÇUK ÇAKMAK

İmza :

.....
(Doç. Dr. Muammer TÜRKÖĞLU)

Bölüm Başkanı

18/06/2026

ETİK İLKE VE KURALLARA UYGUNLUK BEYANNAMESİ

Bu lisans tezinin bana ait, özgün bir çalışma olduğunu; çalışmamın hazırlık, veri toplama, analiz ve bilgilerin sunumu olmak üzere tüm aşamalarında bilimsel etik ilke ve kurallara uygun davrandığımı; bu çalışma kapsamında elde edilen tüm veri ve bilgiler için kaynak gösterdiğimi ve bu kaynaklara kaynakçada yer verdiğimi; bu çalışmanın Samsun Üniversitesi tarafından kullanılan "bilimsel intihal tespit programı"yla tarandığını ve hiçbir şekilde "intihal içermediğini" beyan ederim. Herhangi bir zamanda, çalışmamla ilgili yaptığım bu beyana aykırı bir durumun saptanması durumunda, ortaya çıkacak tüm ahlaki ve hukuki sonuçları kabul ettiğimi bildiririm.

.....
(İmza)

Sidi Mohamed BABA

ÖNSÖZ

Mobil yazılım geliştirme alanı son on yılda hızla genişlemiş; React Native gibi platformlar tek bir kod tabanından birden fazla işletim sistemine uygulama yayımlamayı mümkün kılarak küçük ekiplerin bile büyük ölçekli ürünler üretmesinin önünü açmıştır. Ne var ki, bu ekosistemin sunduğu hızlı üretim imkânı çoğu zaman mimari kararların ertelenmesine, kodun zaman içinde sürdürülebilirliğini kaybetmesine ve teknik borcun kontrolsüz şekilde birikmesine yol açmaktadır.

Bu çalışma kapsamında geliştirilen ArchLens, React Native projelerinde mimari sağlığı statik analiz yöntemleriyle ölçen, problemleri desenleri sayısal göstergeler ve insanca okunur açıklamalarla raporlayan bir araç olarak tasarlanmıştır. Çalışmanın amacı, hem öğrencilere hem de profesyonel ekiplere yönelik, kullanımı kolay ve değerlendirme kriterleri şeffaf bir yardımcı sunmaktır.

Çalışma süresince değerli yönlendirmeleri, sabırlı geri bildirimleri ve akademik rehberliği için danışmanım Sayın Doç. Dr. Nurettin ŞENYER'e içtenlikle teşekkür ederim. Ayrıca lisans öğrenimim boyunca bilgi ve deneyimleriyle bana katkı sağlayan Samsun Üniversitesi Mühendislik Fakültesi Yazılım Mühendisliği Bölümü öğretim üyelerine ve eğitim sürecimde her zaman desteğini hissettiğim aileme şükranlarımı sunarım.

Sidi Mohamed BABA

Haziran, 2026

İÇİNDEKİLER

ÖNSÖZ.....	IV
İÇİNDEKİLER.....	V
ÖZET.....	VII
ABSTRACT.....	IX
TABLolar LİSTESİ.....	XI
ŞEKİLLER LİSTESİ.....	XII
SİMGELER VE KISALTMALAR DİZİNİ.....	XIII
GİRİŞ.....	1
BİRİNCİ BÖLÜM.....	4
KURAMSAL ÇERÇEVE VE İLGİLİ ÇALIŞMALAR.....	4
1.1. Yazılım Mimarisi Kavramı ve Önemi.....	4
1.2. Statik Kod Analizinin Tanımı.....	5
1.3. React Native Geliştirme Ekosistemi.....	6
1.4. Mimari Kötü Kokular ve Teknik Borç.....	7
1.5. İlgili Çalışmalar.....	8
İKİNCİ BÖLÜM.....	9
ARCHLENS SİSTEMİNİN TASARIMI.....	9
2.1. Tasarım Hedefleri ve Gereksinimler.....	9
2.2. Genel Sistem Mimarisi.....	9
2.3. İşlem Boru Hattı (Pipeline).....	10
2.4. Kullanılan Teknolojiler.....	12
ÜÇÜNCÜ BÖLÜM.....	14
GERÇEKLEŞTİRİM (UYGULAMA).....	14
3.1. Girdi Modülü.....	14
3.2. Dosya Tarayıcı.....	14
3.3. AST Çözümleyicisi.....	15
3.4. Katman Sınıflandırıcı.....	15
3.5. Bağımlılık Analizi.....	16
3.6. Kural Motoru.....	16
3.7. Skorlama ve Derecelendirme.....	19
3.8. Raporlama.....	20
3.9. Yapay Zekâ Destekli Açıklamalar.....	20
3.10. Web Arayüzü (Frontend).....	21
DÖRDÜNCÜ BÖLÜM.....	23
DEĞERLENDİRME VE TARTIŞMA.....	23
4.1. Test Stratejisi.....	23

4.2. Örnek Projeler Üzerinde Değerlendirme.....	23
4.3. Performans Ölçümleri.....	24
4.4. Sınırlamalar.....	24
SONUÇ VE ÖNERİLER.....	26
KAYNAKÇA.....	28
EKLER.....	30
ÖZGEÇMİŞ.....	31

ÖZET

ARCHLENS: REACT NATIVE PROJELERİ İÇİN STATİK MİMARİ ANALİZ SİSTEMİ

Sidi Mohamed BABA

Yazılım Mühendisliği

Samsun Üniversitesi

Bilgisayar ve Bilişim Bilimleri Fakültesi

08 Haziran 2026

Danışman: Doç. Dr. Nurettin ŞENYER

React Native, tek kod tabanından birden fazla mobil platforma uygulama üretebilme yeteneği sayesinde günümüzde mobil yazılım geliştirme alanında en çok tercih edilen çatılardan biri hâline gelmiştir. Çatının sunduğu üretkenlik avantajı, ekiplerin kısa sürede çalışan ürünler ortaya koymasını mümkün kılarken; katman ayrımı, dosya boyutu, çevrimsel bağımlılıklar, hook kullanım kuralları ve stil yönetimi gibi mimari kararların ihmal edilmesi durumunda kod tabanlarında ciddi teknik borç birikmesine sebep olmaktadır. Bu çalışmada, söz konusu sorunların geliştirme sürecinin erken aşamalarında saptanmasını sağlayacak bir statik mimari analiz sistemi olan ArchLens tasarlanmış ve gerçekleştirilmiştir.

ArchLens; sıkıştırılmış bir proje arşivini (ZIP), public bir GitHub deposunu veya private bir GitHub deposunu girdi olarak kabul eder. Sistem; Babel tabanlı bir soyut sözdizim ağacı çözümleyicisi, çok sinyalli bir katman sınıflandırıcısı, çevrimsel bağımlılıkları tespit eden bir bağımlılık analizörü ve sekiz adet mimari kuralı yürüten bir kural motoru üzerine kuruludur. Kurallardan elde edilen sonuçlar ağırlıklı bir skorlama mekanizmasıyla birleştirilerek 0–100 aralığında genel bir mimari sağlık puanına ve A–F harf notuna dönüştürülür. İsteğe bağlı olarak başarısız kurallar, Claude büyük dil modeli aracılığıyla proje bağlamına uygun açıklamalarla zenginleştirilir. Sonuçlar hem makine okunabilir JSON hem de geliştirici okunabilir Markdown formatında üretilir; ayrıca tarayıcı tabanlı bir gösterge panelinde görselleştirilir.

Sistem, çalışma kapsamında özel olarak hazırlanan üç test uygulaması (good-app, bad-app ve unusual-layout-app) üzerinde uçtan uca çalıştırılmış; her uygulama için elde edilen sayısal skor önceden manuel olarak belirlenen ground truth ile karşılaştırılmış ve sonuçların ground truth ile tutarlı olduğu doğrulanmıştır. Çalışmanın sonuçları, statik analizin React Native gibi son derece dinamik ve geniş ekosistemlere sahip platformlarda dahi sürdürülebilirlik ölçümü için pratik bir araç olduğunu göstermektedir.

Anahtar Sözcükler: Statik Kod Analizi, Yazılım Mimarisi, React Native, Soyut Sözdizim Ağacı, Mimari Kötü Kokular.

ABSTRACT

ARCHLENS: A STATIC ARCHITECTURAL ANALYSIS SYSTEM FOR REACT NATIVE PROJECTS

Sidi Mohamed BABA

Department of Software Engineering

Samsun University

Faculty of Computer and Information Sciences

08 June 2026

Supervisor: Asst. Prof. Dr. Nurettin ŞENYER

React Native has become one of the most widely adopted frameworks in mobile software development thanks to its ability to ship applications to multiple platforms from a single codebase. While this productivity advantage allows teams to deliver working products quickly, neglecting architectural concerns — layer separation, file size, cyclic dependencies, rules of hooks, style management — frequently leads to the accumulation of significant technical debt. This study presents the design and implementation of ArchLens, a static architectural analysis system whose goal is to detect such issues during the early phases of development.

ArchLens accepts a ZIP archive, a public GitHub repository, or a private GitHub repository as input. The system is built around a Babel-based abstract syntax tree analyzer, a multi-signal layer classifier, a cyclic dependency detector and a rule engine that evaluates eight architectural rules. The outputs of the rules are aggregated through a weighted scoring mechanism and translated into an overall architectural health score in the range 0–100 and a corresponding letter grade between A and F. Optionally, failed rules are enriched with project-aware natural-language explanations produced by the Claude large language model. The results are exported as machine-readable JSON and human-readable Markdown, and are visualised through a browser-based dashboard.

The system has been executed end-to-end on three test applications specially built for the study (good-app, bad-app and unusual-layout-app). The numerical score produced for

each application has been compared against a manually established ground truth, and the results have been found to be consistent with the ground truth. The results suggest that, even in highly dynamic ecosystems such as React Native, static analysis remains a practical instrument for measuring maintainability.

Keywords: Static Code Analysis, Software Architecture, React Native, Abstract Syntax Tree, Architectural Smells.

TABLÖLAR LİSTESİ

Tablo 2. ArchLens'in Karşılaştırıldığı Mevcut Araçlar	8
Tablo 3. Katman Sınıflandırıcısının Kullandığı Sinyaller	16
Tablo 4. Kural Motorundaki Sekiz Mimari Kuralın Özeti	17
Tablo 5. Kuralların Ağırlık ve Önem Derecelerinin Dağılımı	19
Tablo 6. Mimari Sağlık Puanına Karşılık Gelen Harf Notları	19
Tablo 7. Test Projeleri Üzerinde Elde Edilen Sonuçlar	24

ŞEKİLLER LİSTESİ

Şekil 1. ArchLens Genel Sistem Mimarisi	10
Şekil 2. Analiz Boru Hattının Akış Şeması	11
Şekil 3. Markdown Rapor Örneği	20
Şekil 4. Web Gösterge Paneli Genel Görünüm	21

SİMGELER VE KISALTMALAR DİZİNİ

API: Application Programming Interface (Uygulama Programlama Arayüzü)

AST: Abstract Syntax Tree (Soyut Sözdizim Ağacı)

CI: Continuous Integration (Sürekli Bütünleştirme)

CSS: Cascading Style Sheets

DOM: Document Object Model

ECMAScript: European Computer Manufacturers Association Script

HTTP: HyperText Transfer Protocol

JSON: JavaScript Object Notation

JSX: JavaScript XML

LOC: Lines of Code (Kod Satır Sayısı)

SOLID: Tek sorumluluk, Açık/Kapalı, Liskov Yer Değiştirmesi, Arayüz Ayrımı, Bağımlılığın Tersine Çevrilmesi

SQL: Structured Query Language

UI: User Interface (Kullanıcı Arayüzü)

URL: Uniform Resource Locator

GİRİŞ

Mobil uygulamalar, son on beş yıl içinde yazılım sektörünün en hızlı büyüyen alanlarından biri hâline gelmiştir. Statista'nın 2024 yılı verilerine göre yalnızca Google Play Store üzerinde 2,5 milyondan fazla uygulama yer almakta, bunların önemli bir kısmı ise iOS platformunda da paralel olarak yayımlanmaktadır. Yayımlanan uygulamaların büyük bölümü, geçmişte iki ayrı kod tabanı (Android için Kotlin/Java, iOS için Swift/Objective-C) ile geliştirilirken; son yıllarda Facebook tarafından geliştirilen React Native, Google'ın Flutter çatısı ve Microsoft'un .NET MAUI platformu gibi çapraz platform çözümleri ekiplerin önemli bir kısmının tercih ettiği yaklaşım hâline gelmiştir.

React Native özelinde değerlendirildiğinde, çatının sunduğu en önemli avantajların JavaScript/TypeScript ile yazılan tek bir kod tabanından birden fazla platforma yayım yapabilme, yerel kullanıcı arayüzü bileşenlerine doğrudan erişebilme ve geniş bir paket ekosistemine sahip olma olduğu görülmektedir. Buna karşılık React Native, çatı seviyesinde projeye herhangi bir mimari şablon dayatmamakta; klasör düzeninden bileşen ile iş mantığının ayırımına, durum yönetiminden gezinim (navigation) yapılandırmasına kadar geniş bir alanı geliştiriciye bırakmaktadır. Bu özgürlük, deneyimli ekiplerin elinde temiz mimariler üretebilmektedir; ancak öğrencilerin, küçük ekiplerin veya kısa teslim süreleri altında çalışan ekiplerin elinde çoğunlukla iç içe geçmiş bileşenler, çok büyük tek dosyalar, çevrimsel bağımlılıklar ve kontrolsüz şekilde dağılmış stil tanımlarıyla sonuçlanmaktadır.

Söz konusu sorunlar, yazılım mühendisliği literatüründe "mimari kötü kokular" (architectural smells) ve "teknik borç" (technical debt) başlıkları altında uzun yıllardır incelenmektedir. Cunningham (1992), bir kararın bilinçli olarak ertelenmesinin ileride faiziyle birlikte ödenmesi gereken bir borca dönüştüğünü ifade etmiş; sonraki çalışmalar ise teknik borcun yalnızca finansal değil, aynı zamanda psikolojik ve örgütsel bir maliyet de yarattığını ortaya koymuştur. Statik kod analizi araçları, bu maliyetin somut göstergelerini geliştirme sürecinin erken aşamalarında geliştiriciye iletmeye çalışan bir araç ailesidir.

JavaScript ve TypeScript ekosisteminde ESLint, Prettier, SonarQube ve TSLint gibi araçlar yaygın biçimde kullanılmaktadır. Ancak bu araçların büyük bölümü genel amaçlıdır; React veya React Native'in özgün desenlerine (bileşen-ekran ayırımı, navigator yapılandırması, hook kullanım kuralları, yerel API çağrıları gibi) yönelik kurallar yetersiz veya tamamen eksiktir. Bunun yanı sıra mevcut araçlar genellikle satır bazlı uyarılar

retmekte; geliřtiriciye mimarının genel saęlık durumu hakkında tek bakıřta yorumlanabilecek btnleřik bir gsterge sunmamaktadır.

Bu alıřmada, yukarıda tarif edilen bořluęu doldurmak amacıyla ArchLens adı verilen bir statik mimari analiz sistemi tasarlanmış ve gerekleřtirilmiřtir. ArchLens, React Native projelerini hem komut satırı hem de tarayıcı arayz zerinden kabul eden, dosyaları soyut sz dizim aęacı dzeyinde inceleyen, mimari katmanları ok sinyalli bir sınıflandırıcı ile tespit eden ve sekiz mimari kuralın ıřıęında 0–100 aralıęında bir saęlık puanı reten bir aratır. İsteęe baęlı olarak sonuların byk dil modeli ile zenginleřtirilmesi de saęlanmışır; bu sayede bařarısız kurallar projeye zg, doęal dilde aıklamalarla raporlanabilmektedir.

alıřmanın temel arařtırma sorusu řu řekilde zetlenebilir: "React Native projelerinin mimari saęlıęı, statik analiz yntemleriyle, geliřtiricinin gnlk akıřına engel olmayacak bir hızda ve insanca okunur biimde llebilir mi?" Bu soruya yanıt verebilmek iin sırasıyla ařaęıdaki alt amalar belirlenmiřtir:

(i) React Native ekosistemine zg mimari problemleri kataloglamak ve bunlar arasından statik olarak tespit edilebilir olanları semek; (ii) seilen kurallar iin yanlıř-pozitif oranı dřk, performansını kabul edilebilir algoritmalar tasarlamak; (iii) sonuları hem otomatik aralara (JSON) hem de geliřtiricilere (Markdown, web paneli) hitap edecek biimde raporlamak; (iv) sistemin kullanımını kolaylařtıracak basit bir kullanıcı arayz saęlamak; ve (v) elde edilen sonuların geerlilięini gerek dnya projeleri zerinde deęerlendirmek.

alıřma boyunca izlenen yntem  ana ařamadan oluřmaktadır. İlk ařamada literatr taraması yapılmıř; yazılım mimarisi, statik kod analizi, AST tabanlı yntemler, mimari kt kokular ve React/React Native zelinde mevcut aralar incelenmiřtir. İkinci ařamada ArchLens sistemi tasarlanmış ve gerekleřtirilmiřtir; sistem, modler bir boru hattı (pipeline) olarak kurgulanmıřtır. nc ařamada ise sistem, aık kaynaklı React Native řablon projeleri ve zel olarak hazırlanmıř iki sentetik proje zerinde alıřtırılmıř; ıktılar manuel mimari incelemelerle karřılařtırılarak deęerlendirilmiřtir.

alıřmanın katkıları  bařlık altında zetlenebilir. Birincisi, React Native'e zg mimari kurallar literatrdeki genel ESLint kurallarından farklı olarak katmanlar arası iliřkileri ve proje dzeyindeki desenleri esas alacak řekilde tanımlanmıřtır. İkincisi, kuralların yalnızca bireysel olarak deęil, aęırlıklı bir skorlama mekanizmasıyla birleřtirilerek tek bir saęlık puanına dnřtrlmesi saęlanmışır. ncs, sistem byk

dil modelleri ile zenginleştirilmiş; böylece ham kural çıktılarının ötesine geçen, projeye özgü öneriler üretilebilmiştir.

Bu çalışmanın kapsamı; analiz tarafında yalnızca React Native projelerini (Expo, Bare ve TypeScript varyantları dâhil), kullanılan dillerde JavaScript ve TypeScript kaynak dosyalarını kapsamaktadır. Java/Kotlin (Android yerel modülleri) ve Swift/Objective-C (iOS yerel modülleri) tarafları çalışmanın kapsamı dışında bırakılmıştır. Çalışmanın varsayımları arasında "hedef projenin geçerli bir package.json dosyası içerdiği" ve "projenin JavaScript/TypeScript dosyalarının ECMAScript 2022 ve TypeScript 5.x sözdizimine uygun olduğu" yer almaktadır.

Çalışmanın geri kalan kısmı dört ana bölüm hâlinde düzenlenmiştir. Birinci Bölüm, yazılım mimarisi, statik kod analizi ve React Native ekosistemi üzerine kuramsal bir çerçeve sunar ve ilgili çalışmaları kısaca özetler. İkinci Bölüm, ArchLens sisteminin tasarımını; tasarım hedeflerini, mimari kararları, boru hattını ve teknoloji seçimlerini ele alır. Üçüncü Bölüm, sistemin nasıl gerçekleştirildiğini modül modül anlatır. Dördüncü Bölüm, sistemin değerlendirilme biçimini, elde edilen sonuçları, performans ölçümlerini ve sınırlamaları tartışır. Çalışma, sonuç ve gelecek çalışma önerileriyle tamamlanır.

BİRİNCİ BÖLÜM

KURAMSAL ÇERÇEVE VE İLGİLİ ÇALIŞMALAR

1.1. Yazılım Mimarisi Kavramı ve Önemi

Yazılım mimarisi, bir yazılım sisteminin yapısını oluşturan bileşenleri, bu bileşenlerin dışarıdan görülebilen özelliklerini ve aralarındaki ilişkileri tanımlayan üst düzey bir kavramdır (Bass, Clements ve Kazman, 2021: 3). Bu tanım, mimarinin yalnızca "kodun nasıl yazıldığı" sorusuyla değil; aynı zamanda sistemin uzun vadeli değişebilirliği, ölçeklenebilirliği ve sürdürülebilirliği ile ilgili olduğunu vurgulamaktadır. Klasik mühendislik disiplinlerinin aksine yazılım, üretim sonrasında da sürekli olarak değiştirilen bir üründür; bu nedenle mimari kararlar, ürünün toplam yaşam döngüsü boyunca ortaya çıkacak maliyetin en önemli belirleyicilerinden biridir.

Martin (2017: 13), "Clean Architecture" başlıklı çalışmasında yazılım mimarisinin amacını "sistemin geliştirme, devreye alma, çalıştırma ve bakım maliyetlerini en aza indirmek" olarak ifade etmektedir. Bu çerçevede iyi bir mimarinin temel ölçütleri olarak şunlar sıralanmaktadır: bileşenler arasındaki bağımlılıkların yönlendirilebilir olması, soyutlama düzeylerinin tutarlı şekilde katmanlandırılması, yan etkilerin saf işlevsel mantıktan ayrı tutulması ve değişim ihtimali yüksek olan birimlerin değişim ihtimali düşük olan birimlere bağımlı olmaması.

Lehman'ın yazılım evrim yasaları, yazılım sistemlerinin zamanla daha karmaşık hâle geldiğini ve aktif bakım yapılmadığı sürece kullanılabilirliklerinin azaldığını ifade etmektedir (Lehman, 1980: 1062). Lehman'ın ikinci yasası olarak bilinen "artan karmaşıklık yasası" özellikle ilgi çekicidir: bir yazılım sisteminin yapısı, üzerinde herhangi bir karşı önlem alınmadığında, sürekli geliştirme sürecinde giderek bozulmaktadır. Bu yasanın pratik sonucu, mimari sağlığın geliştirme süreci boyunca düzenli olarak ölçülmesi ve gözlemlenen sapmaların düzeltilmesi gerektiğidir.

Modern yazılım geliştirme uygulamasında mimari kararların ölçülmesi için iki temel yaklaşım benimsenmektedir. Bunlardan ilki, manuel mimari incelemelerdir; deneyimli bir mimar veya kıdemli bir geliştirici kodu inceleyerek bir değerlendirme yapar. İkincisi ise otomatik araçların kullanıldığı statik kod analizi yaklaşımıdır. Manuel inceleme, derinlikli ve bağlam farkındalığı yüksek sonuçlar üretebilir; ancak ölçeklenebilir değildir, görece

pahalıdır ve yargı değerlendiriciden değerlendiriciye değişkenlik gösterebilir. Statik analiz ise tutarlı, otomatikleştirilebilir ve sürekli bütünleştirme (CI) süreçlerine entegre edilebilir niteliktedir; bu nedenle çağdaş yazılım mühendisliği uygulamasında iki yaklaşımın birbirini tamamlayıcı şekilde kullanılması yaygın bir uygulamadır.

Bu çalışmada esas alınan mimari ilkeler; Martin'in (2017) tanımladığı SOLID ilkeleri, Fowler'ın (2002) ortaya koyduğu katmanlı mimari modeli ve Evans'ın (2003) "alan-odaklı tasarım" (domain-driven design) yaklaşımı ile uyumludur. Bunlar arasında özellikle "tek sorumluluk ilkesi" (Single Responsibility Principle), "bağımlılığın tersine çevrilmesi" (Dependency Inversion) ve "ayrım ilkesi" (Separation of Concerns) ArchLens'in kurallarının tasarımında doğrudan rol oynamıştır.

1.2. Statik Kod Analizinin Tanımı

Statik kod analizi, programın çalıştırılmadan kaynak kodunun veya derlenmiş ürününün incelenmesi sonucu çeşitli özelliklerin tespit edilmesi sürecidir (Nielson, Nielson ve Hankin, 2005: 1). "Statik" sıfatı, analizin programın yürütülmesinden bağımsız olarak yapılmasını ifade etmek üzere; dinamik analizden ayırt etmek için kullanılır. Dinamik analiz, programın belirli girdilerle çalıştırılması sonucu davranışsal özelliklerin gözlemlenmesini esas alırken; statik analiz, kodun olası tüm yürütme yolları üzerinde teorik bir değerlendirme yapar.

Statik analiz araçlarının pratik kullanım alanları oldukça geniştir. Tipik kullanım senaryoları arasında; söz dizim hatalarının tespiti, kullanılmayan değişkenlerin saptanması, olası null dereference durumlarının uyarı olarak raporlanması, güvenlik açıklarının (SQL injection, cross-site scripting gibi) öngörülmesi, performans sorunlarına yol açabilecek desenlerin işaretlenmesi ve mimari kuralların ihlal edildiği noktaların belirlenmesi yer almaktadır. Ayewah, Pugh, Hovemeyer, Morgenthaler ve Penix (2008), FindBugs aracı üzerinde yaptıkları çalışmada statik analiz uyarılarının önemli bir kısmının gerçek hata olduğunu, ancak yanlış-pozitif oranının kullanıcı deneyimi açısından kritik bir etken olduğunu rapor etmiştir.

Johnson, Song, Murphy-Hill ve Bowdidge (2013), geliştiricilerin statik analiz araçlarını neden tercih etmediği üzerine yaptıkları nitel çalışmada üç ana neden tespit etmiştir: araçların ürettiği uyarıların geliştirici için anlaşılır olmaması, yanlış-pozitiflerin yüksek olması ve sonuçların geliştirme akışına uygun biçimde sunulmaması. Bu bulgu, ArchLens'in tasarımını doğrudan etkilemiştir; sistem, yalnızca kural ihlallerini tespit etmekle

kalmamakta, her ihlali tek bir yerleşik açıklamayla ve isteğe bağlı olarak büyük dil modeli üretimi proje bağlamı açıklamasıyla birlikte sunmaktadır.

Statik analiz yöntemleri kullandıkları soyutlama düzeyine göre üç sınıfta incelenebilir. İlk sınıf, basit metinsel desen eşleştirme (regex tabanlı) yaklaşımdır; kolayca uygulanabilir olmakla birlikte yanlış-pozitif oranı yüksektir. İkinci sınıf, sözcüksel (lexer) çıktılarını kullanan analizlerdir; söz dizimine kısmi farkındalık sağlar. Üçüncü sınıf ise soyut sözdizim ağacı veya kontrol akış grafiği gibi daha üst düzey temsiller üzerinde çalışan analizlerdir; en yüksek doğruluğu sunmakla birlikte uygulanması daha karmaşıktır. ArchLens, üçüncü sınıfa giren bir araçtır.

1.2.1. Soyut Sözdizim Ağacı (AST) Temelli Analiz

Soyut sözdizim ağacı; bir programın kaynak kodunun, programlama dilinin gramer kurallarına göre çözümlenmesi sonucu oluşturulan ağaç biçimli soyut temsildir. AST, kaynak kodu bir veri yapısına dönüştürerek programatik olarak gezilmesini, incelenmesini ve değiştirilmesini mümkün kılar. JavaScript/TypeScript ekosisteminde en yaygın kullanılan AST üreticilerinden ikisi Babel ve TypeScript derleyicisidir (tsc); ArchLens, esnekliği ve eklenti ekosisteminin genişliği nedeniyle Babel'i tercih etmektedir.

AST tabanlı analizin temel avantajı, kodun anlamsal yapısına doğrudan erişim sağlamasıdır. Örneğin bir bileşenin hangi modüllerden hangi sembolleri içe aktardığı, hangi fonksiyon çağrılarını yaptığı, hangi JSX elemanlarını oluşturduğu ve hangi hook'ları çağırdığı sorularına AST üzerinde basit gezintilerle yanıt verilebilir. Bu sayede yüzeysel desen eşleştirmeyle elde edilemeyecek düzeyde hassas sonuçlar üretmek mümkündür.

1.3. React Native Geliştirme Ekosistemi

React Native, 2015 yılında Facebook (günümüzdeki adıyla Meta) tarafından açık kaynak olarak yayımlanan bir mobil uygulama geliştirme çatısıdır. React Native, React kütüphanesinin bileşen modelini ve durum yönetimi felsefesini benimser; ancak tarayıcı tabanlı bir DOM yerine, iOS ve Android'e özgü yerel kullanıcı arayüzü bileşenleri üzerinde çalışır. Bu sayede yazılan kodun büyük bir kısmı her iki platformda da paylaşılabilir; yerel bileşenlere doğrudan erişim ise platforma özgü modüller aracılığıyla sağlanır.

React Native ekosistemi içerisinde geliştiriciler için iki temel proje şablonu yaygın olarak kullanılmaktadır. Bunlardan ilki "Bare React Native" olarak bilinen, geliştiricinin tüm yerel proje yapılandırmasına (Gradle, Xcode, iOS/Android klasörleri) doğrudan erişimi

olduğu şablondur. İkincisi ise Expo ekibinin geliştirdiği yönetimli iş akışıdır ("Managed Workflow"); bu yaklaşımda yerel yapılandırma soyutlanmış ve geliştiriciye yalnızca JavaScript/TypeScript tarafıyla ilgilenmesine olanak sağlayan bir araç zinciri sunulmuştur. Her iki şablon da ArchLens tarafından desteklenmektedir.

React Native uygulamaları yapısal olarak çeşitli mimari katmanlardan oluşur. Tipik bir uygulamada en az şu katmanlar bulunmaktadır: kullanıcı arayüzü bileşenleri (UI components), ekran düzeyindeki konteynerler (screens), gezinim yapılandırması (navigation), durum yönetimi (state), iş mantığı ve uzak servislerle iletişim katmanı (services), yardımcı işlevler (utilities) ve özel bağlamlar (contexts). Bu katmanların görece bağımsız tutulması, kodun sürdürülebilirliği açısından kritik öneme sahiptir; ancak React Native'in proje şablonu seviyesinde katmanlama dayatmaması nedeniyle uygulamada katmanlar arası ihlaller sıkça görülmektedir.

React, geliştiricilere durum ve yan etki yönetimi için "hook" adı verilen bir soyutlama sunmaktadır. Hook'lar bileşenin yaşam döngüsünde belirli koşullarla çağrılmaları gerektiğinden, Meta tarafından "Hook Kullanım Kuralları" (Rules of Hooks) yayımlanmıştır. Bu kurallara göre hook'lar yalnızca fonksiyon bileşenlerinin veya başka hook'ların üst düzeyinde, koşul veya döngü içerisinde olmaksızın çağrılmalıdır. Kuralların ihlali, çalışma zamanında çok zor teşhis edilebilen hatalara yol açabilir; bu nedenle ArchLens, bu kuralları statik düzeyde denetleyen bir kurala sahiptir.

1.4. Mimari Kötü Kokular ve Teknik Borç

Mimari kötü kokular, Fowler ve Beck'in (1999) ortaya koyduğu "kod kokusu" kavramının mimari düzeye uyarlanmış hâlidir. Garcia, Popescu, Edwards ve Medvidovic (2009), mimari düzeyde dört temel kötü koku tanımlamıştır: hub-benzeri bağımlılık (Hub-Like Dependency), katı çevrim (Cyclic Dependency), dağınık katmanlanma (Scattered Functionality) ve karara aykırılık (Decision Violation). Bu kötü kokular, ArchLens'in kural tasarımında dikkate alınmıştır.

Teknik borç kavramı ise Cunningham (1992) tarafından bilinçli olarak yapılan tasarım uzlaşmalarının zaman içinde finansal bir borç gibi birikmesini ifade etmek üzere kullanılmıştır. Kruchten, Nord ve Ozkaya (2012), teknik borcu "görünür ve görünmez", "kasıtlı ve kasıtsız" olarak çapraz biçimde dört bölüme ayırmış; statik analiz araçlarının özellikle "görünmez ve kasıtsız" borcun tespit edilmesinde etkili olduğunu vurgulamıştır.

1.5. İlgili Çalışmalar

JavaScript/TypeScript ekosisteminde statik analiz aracı olarak en yaygın kullanılan iki sistem ESLint ve SonarQube'dur. ESLint, eklenti tabanlı mimarisi ve geniş kural kütüphanesi ile öne çıkmakta, projenin kendi kurallarının kolayca yazılabılmesine imkân tanımaktadır. SonarQube ise daha kurumsal odaklı bir araçtır; metrik koleksiyonu, geçmiş eğilim analizi ve birden fazla dile destek sunmaktadır. Her iki araç da satır bazlı uyarılar üretmek üzerine tasarlanmıştır; mimari düzey raporlama açısından sınırlıdır.

Akademik literatürde, JavaScript ekosistemine özgü mimari analiz çalışmaları görece azdır. Silva, Tsantalis ve Valente (2016), JavaScript projelerinde tasarım kötü kokularını tespit eden bir aracın prototipini sunmuş; ancak React veya React Native gibi özel çatılara odaklanmamışlardır. React özelinde, Mannan, Ahmed, Almurshed, Dig ve Jensen (2016), Android uygulamalarındaki kötü kokuları incelemiş; React Native odaklı bir çalışmaya rastlanmamıştır. Tablo 2'de mevcut araçların ArchLens ile karşılaştırması özetlenmektedir.

Tablo 2. ArchLens'in karşılaştırıldığı mevcut araçlar.

Araç	Hedef Çatı	Mimari Skor	Açıklama Üretimi	Web Paneli
ESLint	Genel JS/TS	Yok	Statik mesaj	Yok
SonarQube	Çok dilli	Genel/satır	Statik mesaj	Var
TSLint (arşivlendi)	TypeScript	Yok	Statik mesaj	Yok
JSHint	JavaScript	Yok	Statik mesaj	Yok
ArchLens	React Native	0–100 + A–F	LLM destekli	Var

Görüldüğü üzere ArchLens, mevcut araçlardan üç temel açıdan ayrılmaktadır. Birincisi, hedef çatının React Native olarak özelleştirilmiş olmasıdır; bu sayede kurallar gerçekten uygulanabilir desenler üzerine kurulmuştur. İkincisi, yalnızca satır düzeyinde uyarılar üretmek yerine bütünlük bir mimari sağlık puanı raporlanmasıdır. Üçüncüsü, başarısız kurallar için isteğe bağlı olarak büyük dil modeli destekli proje bağlamı açıklaması üretilmesidir; bu son özellik, literatürde benzer bir araca rastlanmamış olması nedeniyle özgün bir katkı niteliği taşımaktadır.

İKİNCİ BÖLÜM

ARCHLENS SİSTEMİNİN TASARIMI

2.1. Tasarım Hedefleri ve Gereksinimler

ArchLens sisteminin tasarımına başlanırken, hem işlevsel hem de işlevsel olmayan gereksinimlerin açık biçimde belirlenmesine özen gösterilmiştir. İşlevsel gereksinimler şu şekilde özetlenebilir: sistem, sıkıştırılmış bir proje arşivini (ZIP), public bir GitHub deposunu veya private bir GitHub deposunu girdi olarak kabul etmeli; bu girdiyi geçici bir çalışma alanına hazırlamalı; hazırlık sonrasında dosyaları tarayıp mimari katmanlara sınıflandırmalı; bağımlılık çevrimlerini tespit etmeli; tanımlı mimari kuralları yürütmeli; sonuçları ağırlıklı bir skor mekanizmasıyla birleştirmeli; ve son olarak makine ile insan tarafından okunabilen iki ayrı formatta rapor üretmelidir.

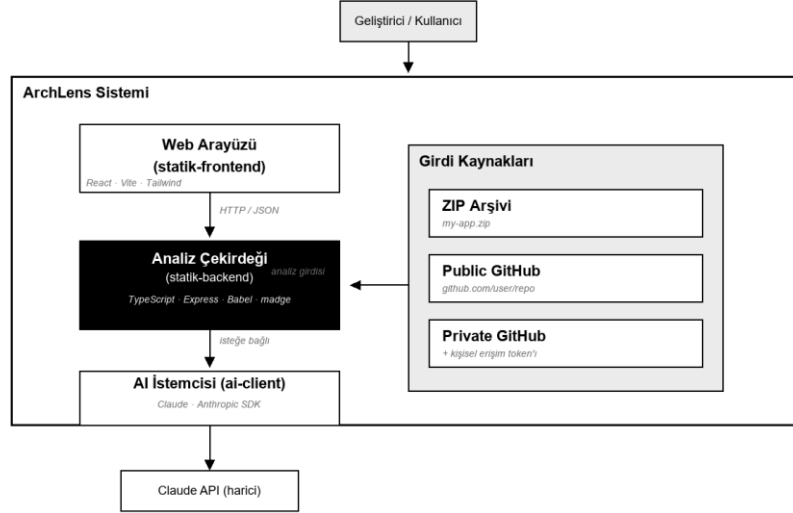
İşlevsel olmayan gereksinimler ise üç ana ekseninde toplanmıştır. Performans ekseninde, sistemin orta ölçekli bir projeyi saniyeler içinde, geliştiricinin akışını bozmayacak biçimde tamamlaması hedeflenmiştir. Genişletilebilirlik ekseninde, yeni kuralların veya yeni katman sinyallerinin sisteme eklenmesi en fazla birkaç saatlik bir geliştirme eforu gerektirmelidir. Güvenilirlik ekseninde ise hatalı veya React Native olmayan projeler tespit edilmeli ve bir analiz yapmadan önce kullanıcıya açık bir hata mesajı sunulmalıdır; sistemin sessizce hatalı puan üretmesi en büyük başarısızlık modu olarak kabul edilmiştir.

Tasarım hedeflerinin en kritik olanı, yanlış-pozitif oranının düşürülmesi olarak belirlenmiştir. Johnson ve diğerlerinin (2013) bulguları, yanlış-pozitif oranının yüksek olmasının statik analiz araçlarından vazgeçilmesinin en önemli nedeni olduğunu göstermektedir. Bu nedenle ArchLens'in kurallarının her biri için bir "güven düzeyi" (confidence level) hesaplanmakta; düşük güvenli bulgular kullanıcıya farklı bir ağırlıkla iletilmektedir.

2.2. Genel Sistem Mimarisi

ArchLens, monorepo şeklinde organize edilmiş birden fazla paketten oluşan bir sistemdir. Tasarımın özünde, sorumlulukların birbirinden ayrıştırılmış paketlere dağıtıldığı modüler bir yaklaşım yer almaktadır. Bu yaklaşım, her paketin bağımsız olarak test

edilebilmesine, geliştirilebilmesine ve gerektiğinde yeniden kullanılabilmesine imkân tanımaktadır. Şekil 1, sistemin genel mimari şemasını göstermektedir.



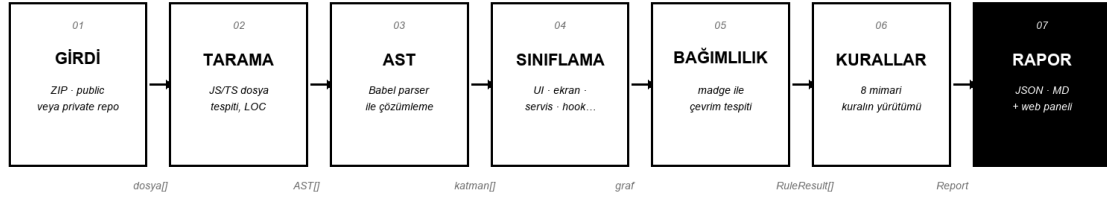
Şekil 1. ArchLens genel sistem mimarisi.

Sistemin başlıca üç paketi bulunmaktadır. İlki, tüm analiz mantığını barındıran "statik-backend" paketidir; bu paket hem komut satırı arayüzü olarak doğrudan kullanılabilir, hem de bir HTTP sunucusu olarak çalıştırılabilir. İkincisi, web tarayıcısı üzerinden çalışan ve analiz sonuçlarını görselleştiren "statik-frontend" paketidir. Üçüncüsü ise büyük dil modeli ile iletişim kuran ortak "ai-client" paketidir; bu paket isteğe bağlı olarak başarısız kurallar için açıklama üretmekte kullanılır.

Backend paketinin iç mimarisi de modüler bir biçimde tasarlanmıştır. Paket; girdi (input), tarama (scanner), AST çözümlemesi (ast), katman sınıflandırması (classifier), bağımlılık analizi (dependency), kural motoru (rules), skoreleme (scoring), raporlama (reporting) ve yapay zekâ zenginleştirme (ai) olmak üzere dokuz alt modüle ayrılmıştır. Her alt modül, kendisinden önceki modülün ürettiği veriyi tüketmekte ve bir sonraki modüle aktarmaktadır; bu sayede klasik boru hattı (pipeline) mimarisi kurulmaktadır.

2.3. İşlem Boru Hattı (Pipeline)

Sistemin işlem boru hattı, girdiden raporlamaya kadar sıralı biçimde yürütülen yedi temel aşamadan oluşmaktadır. Şekil 2, bu aşamaları görsel olarak özetlemektedir. Her aşama, kendisinden önceki aşamanın ürettiği yapı taşlarını kullanmakta ve bir sonraki aşamaya devretmektedir; arada herhangi bir paylaşılan küresel durum (global state) tutulmamaktadır.



Her aşama bir önceki aşamanın çıktısını tüketir, paylaşılan küresel durum kullanılmaz.

Şekil 2. Analiz boru hattının akış şeması.

İlk aşamada "girdi modülü" devreye girmekte ve kullanıcı tarafından sağlanan kaynağı geçici bir çalışma dizinine hazırlamaktadır. Bir ZIP arşivi yüklendiğinde modül dosyayı geçici bir klasöre açmakta; public veya private bir GitHub URL'i verildiğinde ise depo, geçici bir dizine klonlanmaktadır (private depolar için kullanıcının sağladığı kişisel erişim belirteci kullanılır). Bu aşamanın çıktısı "PreparedSource" adı verilen bir nesnedir; nesne, çalışma dizininin yolunu ve analiz tamamlandığında çağrılacak "cleanup" fonksiyonunu içermektedir.

İkinci aşama "dosya tarayıcı" aşamasıdır. Tarayıcı, hazırlanmış çalışma dizinini özyinelemeli biçimde gezerek JavaScript ve TypeScript dosyalarını tespit etmektedir. Bu sırada node_modules, dist, build ve benzeri klasörler hariç tutulmaktadır. Tarayıcı ayrıca projenin package.json dosyasını okumakta ve alt proje (örneğin packages/ altındaki paketler) algıladığında bunları ayrı bir listede tutmaktadır.

Üçüncü aşamada "AST çözümleyicisi" çalışmakta ve her dosyanın Babel ile ayrıştırılmış soyut sözdizim ağacı üzerinden istatistiksel bilgiler toplamaktadır. Bu bilgiler arasında JSX kullanımı, içe ve dışa aktarma ifadeleri, çağrılan hook'lar, navigator ve context yaratıcı çağrılar ile yerel API erişimleri yer almaktadır. Çıktı "AstFacts" adı verilen bir kayıt yapısıdır ve dosya yolundan ilgili dosyanın AST gerçekliklerine bir eşleme tutmaktadır.

Dördüncü aşama "katman sınıflandırıcısı"dır. Bu aşamada her dosyaya, AST gerçekliklerinin ve isteğe bağlı kullanıcı yapılandırmasının (.evaluator.json) ışığında bir mimari katman etiketi atanmaktadır. Mümkün etiketler arasında "ui", "screen", "navigation", "service", "hook", "context", "util" ve "unknown" yer almaktadır. Sınıflandırıcı, birden fazla

sinyale dayalı bir puanlama yapısı kullanarak en yüksek puanı alan etiketi atamakta; kararsız kaldığı dosyaları "unknown" olarak işaretlemektedir.

Beşinci aşama "bağımlılık analizi"dir. Bu aşamada madge kütüphanesi kullanılarak projenin modül bağımlılık grafiği oluşturulmakta ve çevrimsel bağımlılıklar tespit edilmektedir. Çıktı, çevrimleri oluşturan dosya listelerinden oluşmaktadır. Bu bilgi hem rule2 (çevrimsel bağımlılık kuralı) tarafından doğrudan, hem de rule4 (katman ayrımı kuralı) tarafından dolaylı olarak kullanılmaktadır.

Altıncı aşama "kural motoru"dur. Motorda, sırasıyla aşağıdaki kurallar çalıştırılır: rule1 servis katmanı varlığı, rule2 çevrimsel bağımlılık tespiti, rule3 dosya boyutu kontrolü, rule4 katman ayrımı, rule5 hook kullanım kuralları, rule6 satır içi stil tespiti, rule7 isimlendirme tutarlılığı ve rule8 yerel API kullanımının UI katmanında yer alıp almadığı kontrolü. Her kural; bir başarı/başarısızlık etiketi, bir güven düzeyi, ihlallerin yer aldığı dosyaların listesi ve sabit bir açıklama metnini içeren "RuleResult" nesnesi döndürmektedir.

Yedinci aşama, "skorlama ve raporlama" olarak iki adımdan oluşmaktadır. Skorlayıcı, her kuralın sonucunu kuralın önem ağırlığıyla çarparak toplamakta ve 0–100 aralığında bir genel skor üretmektedir. Bu skor, sonraki adımda eşik değerleriyle karşılaştırılarak A–F arasında bir harf notuna dönüştürülür. Raporlayıcı ise sonuçları hem makine okunabilir JSON formatında, hem de geliştirici okunabilir Markdown formatında üretmektedir.

2.4. Kullanılan Teknolojiler

Sistemin tasarımında, hem ekosistem olgunluğu hem de geliştirme verimliliği göz önünde bulundurularak teknoloji seçimi yapılmıştır. Backend paketi, tip güvenliği sağlamak amacıyla TypeScript ile geliştirilmiştir. Ayrıştırma için Babel ekosisteminin parçası olan @babel/parser kütüphanesi kullanılmıştır; bunun gerekçesi Babel'in JSX ve TypeScript sözdizimine yerleşik destek vermesi ve geniş bir eklenti kütüphanesine sahip olmasıdır. Modül bağımlılığı analizi için olgun ve yaygın olarak kullanılan madge kütüphanesi tercih edilmiştir.

HTTP sunucusu olarak, hafif yapısı ve geniş middleware ekosistemi nedeniyle Express kullanılmıştır. Dosya yüklemeleri için multer ara katman yazılımı, CORS desteği için cors

ara katmanı tercih edilmiştir. Sıkıştırılmış proje dosyalarını açmak için adm-zip kütüphanesi kullanılmıştır. Komut satırı argümanlarının çözümlenmesi commander kütüphanesi, ilerleme göstergeleri ise ora kütüphanesi ile gerçekleştirilmiştir.

Frontend paketi, modern ve hızlı geliştirme deneyimi sağlayan React, Vite ve Tailwind CSS üçlüsü ile inşa edilmiştir. React, bileşen tabanlı kullanıcı arayüzü inşası için endüstri standardı bir kütüphanedir; Vite, geliştirme sırasında çok hızlı yeniden yükleme süresi sunan modern bir paketleyicidir; Tailwind ise yardımcı sınıf tabanlı bir CSS çatısı olarak tutarlı bir görsel dil sağlamaktadır.

Yapay zekâ zenginleştirmesi için Anthropic'in resmî TypeScript SDK'sı üzerinden Claude modeli kullanılmıştır. Modelin seçimindeki temel faktörler; uzun bağlam penceresi sayesinde tüm rapor sonuçlarının tek bir istek olarak iletilebilmesi, ürettiği yanıtların tutarlılığı ve maliyet ile cevap kalitesi arasındaki dengedir. Çağrılar, isteğe bağlı bir analiz parametresi tarafından tetiklenmekte; sistemin temel işlevselliği yapay zekâ olmadan da tam olarak çalışmaktadır.

ÜÇÜNCÜ BÖLÜM

GERÇEKLEŞTİRİM (UYGULAMA)

3.1. Girdi Modülü

Girdi modülü, ArchLens'in dış dünyaya açılan ilk kapısıdır. Üç farklı kaynak türü için ayrı işleyiciler içerir: yüklenen sıkıştırılmış dosya (ZipSource), public GitHub deposu (PublicGithubSource) ve private GitHub deposu (PrivateGithubSource). Her işleyici, ortak "PreparedSource" arayüzünü uygulamakta; bu arayüz "rootPath", "rootName" ve asenkron bir "cleanup" fonksiyonu sunmaktadır.

ZIP işleyicisi en yaygın kullanılan kaynaktır; gelen sıkıştırılmış dosyayı geçici bir klasöre açar ve cleanup fonksiyonu bu geçici klasörü kaldırır. Public GitHub işleyicisi, verilen URL'yi git komutu ile klonlar; klonlama sığ ("shallow clone") yapıldığından büyük depolar için bile makul sürelerde tamamlanmaktadır. Private GitHub işleyicisi de aynı şekilde çalışmakta, ek olarak kullanıcının sağladığı kişisel erişim belirtecini (personal access token) kimlik doğrulama amacıyla kullanmaktadır.

Girdi hazırlandıktan sonra önemli bir doğrulama adımı yapılır: hedef projenin gerçekten bir React Native projesi olup olmadığı tespit edilir (detect-rn.ts). Doğrulama; package.json dosyasındaki bağımlılıkların incelenmesi, Metro konfigürasyon dosyasının varlığı, Expo'ya özgü app.json/app.config.js dosyalarının kontrolü ve android/ile ios/ klasörlerinin varlığı gibi çoklu sinyalin değerlendirilmesi yoluyla gerçekleştirilir. Hiçbir sinyal eşleşmediğinde kullanıcıya açıklayıcı bir hata mesajı döndürülür ve analiz tamamen iptal edilir; bu sayede React Native dışı projeler üzerinde anlamsız puan üretilmesi engellenir.

3.2. Dosya Tarayıcı

Dosya tarayıcı, çalışma dizinini özyinelemeli olarak gezer ve analiz edilecek kaynak kod dosyalarının listesini hazırlar. Hedef uzantılar .js, .jsx, .ts ve .tsx olarak belirlenmiştir. Tarayıcı, gereksiz analizden kaçınmak için aşağıdaki klasörleri otomatik olarak hariç tutar: node_modules, dist, build, .git, coverage, ios, android ve __tests__ alt klasörleri. Bu listeye ek olarak kullanıcı, .evaluator.json üzerinden kendi yoksayma listesini de tanımlayabilmektedir.

Tarayıcı, her dosya için yalnızca yolu değil; aynı zamanda satır sayısını da (LOC) ölçer. Bu ölçüm, sonraki aşamalarda rule3 (dosya boyutu kuralı) tarafından kullanılır. Tarayıcı çıktısı "ScannedFile[]" tipinden bir dizi olup her eleman dosyanın yolunu, içeriğini, LOC değerini ve dosya boyutunu içerir.

Tarayıcı ayrıca "monorepo" desenini tespit etmek üzere tasarlanmıştır. Eğer kök dizin altında packages/ veya apps/ klasörleri bulunuyorsa ve bunların altında alt-package.json'lar varsa, tarayıcı bunları "alt projeler" olarak tespit eder ve kullanıcıya hangi alt projelerin analiz dışı bırakıldığını raporlar. Bu özellik, çok paketli depolarda yanlış skor üretiminin önüne geçmektedir.

3.3. AST Çözümleyicisi

AST çözümleyicisi modülü, her bir taranan dosyanın Babel ile ayrıştırılmış soyut sözdizim ağacı üzerinde gezinerek ileri aşamaların ihtiyaç duyacağı gerçeklikleri çıkarır. Çıkarılan gerçeklikler arasında: dosyanın JSX içerip içermediği, varsayılan dışa aktarımın bir bileşen olup olmadığı, çağrılan tüm hook isimleri ve bunların koşulsal yapı içinde olup olmadığı, dosyanın çağırdığı navigator yaratıcı fonksiyonları (createStackNavigator, createBottomTabNavigator vb.), createContext çağrıları ile yerel API erişimleri (Platform, NativeModules, PixelRatio gibi) yer alır.

Çözümleyici, Babel'in @babel/traverse modülünü kullanarak ağacı tek geçişle (single-pass) ziyaret etmekte; bu sayede dosya başına maliyet doğrusal düzeyde tutulmaktadır. Karşılaşılan her ilgi düğümünde uygun bir işleyici tetiklenmekte ve çıktı tek bir "AstFacts" nesnesinde biriktirilmektedir. Hatalı ayrıştırma (SyntaxError) durumunda dosya atlanmakta ancak bir uyarı ile birlikte rapora eklenmektedir; bu yaklaşım sayesinde tek bir hatalı dosya, tüm analizi engellememektedir.

Modülün önemli bir tasarım kararı, AST gerçekliklerinin sınıflandırıcıdan önce hesaplanmasıdır. Bu sıra; sınıflandırıcının pek çok kararı (JSX varlığı, navigator çağrıları, createContext gibi) AST gerçekliklerinden okuduğu için zorunludur. Boru hattının önceki versiyonlarında sınıflandırıcı AST'ten önce çalıştığında, salt dosya adı ve klasör konumu sinyallerine dayanan kararların yanlış-pozitif oranı belirgin biçimde yüksek bulunmuştur.

3.4. Katman Sınıflandırıcı

Katman sınıflandırıcısı, ArchLens'in en karmaşık modüllerinden biridir. Her dosya için, mümkün her etiket adayı ("ui", "screen", "navigation", "service", "hook", "context",

"util", "unknown") üzerinde bir puan hesaplanır ve en yüksek puanı alan etiket dosyaya atanır. Puanlama, Tablo 3'te özetlenen sinyallerin ağırlıklı toplamı şeklindedir.

Tablo 3. Katman sınıflandırıcısının kullandığı sinyaller.

Sinyal	Açıklama	Etki
Klasör adı	/screens, /services, /hooks gibi standart klasör isimleri	Yüksek
Dosya adı	useFoo.ts (hook), FooScreen.tsx (ekran)	Orta
JSX varlığı	Dosyada JSX ifadesi bulunması	Yüksek (UI)
Hook çağrıları	useState, useEffect, useReducer çağrıları	Orta
createContext	React Context yaratma çağrısı	Yüksek (context)
Navigator çağrıları	createStackNavigator vb.	Yüksek (navigation)
Yerel API erişimi	Platform, NativeModules, PixelRatio kullanımı	Düşük
Varsayılan dışa aktarım	Fonksiyon bileşeni döndüren default export	Orta

Kullanıcı, projenin kendine özgü klasör isimlendirme alışkanlıklarını .evaluator.json dosyası üzerinden bildirebilmektedir. Örneğin bir proje ekranlarını /pages klasörü altında topluyorsa, kullanıcı bu işlemeyi yapılandırma dosyasına ekleyerek sınıflandırıcının doğru davranmasını sağlayabilir.

3.5. Bağımlılık Analizi

Bağımlılık analizi modülü, madge kütüphanesinin programatik API'sini kullanarak projenin modül bağımlılık grafiğini oluşturmakta ve bu grafik üzerinde çevrimsel bağımlılıkları (cyclic dependencies) tespit etmektedir. Madge, kullanılan TypeScript yapılandırmasını otomatik algılayan ve hem import hem de require ifadelerini doğru biçimde işleyen olgun bir kütüphanedir.

Modülün çıktısı, çevrim oluşturan dosya listelerinin yer aldığı bir dizi şeklindedir. Örneğin "foo.ts -> bar.ts -> baz.ts -> foo.ts" bir çevrim olarak tek bir liste hâlinde rapora yansıtılır.

Çevrim tespiti, performans açısından maliyetli bir işlemdir; ancak madge'nin kullandığı Tarjan algoritması doğrusal aralıkta çalışmakta ve orta ölçekli projelerde toplam analiz süresine kayda değer bir etki yapmamaktadır.

3.6. Kural Motoru

Kural motoru, ArchLens'in mimari değerlendirme yapan çekirdek bileşenidir. Motor; ortak bir "Rule" arayüzünü uygulayan sekiz kural fonksiyonunu sırasıyla çalıştırır. Her kural, "Project" tipinden bir parametre alır ve "RuleResult" tipinde bir nesne döndürür. RuleResult nesnesi; kuralın benzersiz kimliği, insan tarafından okunabilir adı, başarı durumu, güven düzeyi, ağırlığı, açıklama metni ve ihlal listesi alanlarını içerir. Tablo 4, sekiz kuralın amaçlarını özetlemektedir.

Tablo 4. Kural motorundaki sekiz mimari kuralın özeti.

No	Kural Adı	Amaç
1	Servis Katmanı Varlığı	İş mantığı ve API çağrıları için ayrı bir katmanın var olup olmadığı
2	Çevrimsel Bağımlılık	Modüller arasında çevrim bulunmaması
3	Dosya Boyutu	Dosyaların 400 LOC eşiğini aşmaması
4	Katman Ayrımı	UI'dan servise, servisten UI'a doğrudan bağımlılık olmaması
5	Hook Kullanım Kuralları	useX çağrılarının koşulsuz ve üst düzeyde olması
6	Satır İçi Stil	JSX içinde aşırı miktarda inline style kullanılmaması
7	İsimlendirme	Bileşen, ekran ve hook dosyalarının tutarlı isimlendirilmesi
8	UI'da Yerel API	Platform, NativeModules gibi API'lerin servis katmanında olması

3.6.1. Rule 1 — Servis Katmanı Varlığı

Bu kural, projede en az bir tanımlı servis modülünün bulunmasını şart koşar. Servis modülü, dış servislerle iletişim kuran veya iş mantığı barındıran ve UI katmanından bağımsız olarak test edilebilen bir modül olarak tanımlanır. Kural, sınıflandırıcı tarafından "service" olarak etiketlenmiş dosyaların sayısını kontrol eder. Sıfır olduğunda kural başarısız; bir veya daha fazla olduğunda başarılı sayılır. Bu kuralın gerekçesi, küçük projelerde dahi iş mantığının UI katmanından ayrıştırılmasının uzun vadeli sürdürülebilirliği belirgin biçimde artırdığı şeklindedir (Martin, 2017: 95).

3.6.2. Rule 2 — Çevrimsel Bağımlılık

Bu kural, bağımlılık analizi modülünden alınan çevrim listesinin boş olup olmadığını kontrol eder. Çevrim sayısı arttıkça kural sonucunun güven düzeyi azalır; örneğin bir veya iki küçük çevrim toleransla karşılanabilirken on ve üzeri çevrimin bulunması projenin

bağımlılık yapısının ciddi şekilde bozulduğunu göstermektedir. Çevrimler, yalnızca toplam sayı olarak değil, dosya listeleriyle birlikte rapora eklenir; böylece geliştirici hangi modüllerin sorunlu olduğunu doğrudan görebilmektedir.

3.6.3. Rule 3 — Dosya Boyutu

Bu kural, projedeki tüm dosyaların LOC değerini kontrol eder ve önceden tanımlanmış 400 LOC eşikini aşan dosyaları ihlal olarak işaretler. Eşik değeri, literatürde dosya başına 200–500 satır arasında öneriler bulunmasına rağmen React Native bileşenlerinin sıklıkla stil tanımlamalarıyla şişme eğilimi göz önünde bulundurularak 400 olarak belirlenmiştir. Kural sonucu, ihlal edilen dosya sayısının toplam dosya sayısına oranına göre derecelendirilmektedir.

3.6.4. Rule 4 — Katman Ayrımı

Bu kural, sınıflandırıcı tarafından etiketlenmiş dosyaların import ifadelerini analiz ederek katmanlar arası ihlalleri tespit eder. İhlal sayılan durumlar: (i) bir UI bileşeninin doğrudan bir API istemcisi servisten import etmesi; (ii) bir servis modülünün bir UI bileşeninden import etmesi; (iii) bir hook modülünün bir ekrandan import etmesi. Bu kural, Martin'in (2017) ortaya koyduğu "bağımlılığın tersine çevrilmesi" ilkesinin doğrudan bir uygulamasıdır.

3.6.5. Rule 5 — Hook Kullanım Kuralları

Bu kural, React'in resmi "Rules of Hooks" belgesinde tanımlanan iki temel kuralı doğrular: (a) hook'lar yalnızca fonksiyon bileşenlerinin veya başka hook'ların üst seviyesinde çağrılmalıdır, (b) hook'lar koşullu yapıların (if, for, switch) içinde çağrılmamalıdır. AST çözümleyicisinin sağladığı "hook çağrı listesi" bilgisi, koşullu yapı içeren çağrıların ihlal olarak işaretlenmesini sağlar. Bu kuralın varsayılan ağırlığı yüksektir; çünkü ihlali çalışma zamanında tespit edilmesi güç hatalara neden olmaktadır.

3.6.6. Rule 6 — Satır İçi Stil

Bu kural, JSX içerisinde doğrudan stil objesi atamalarının sayısını sınırlar. Çok sayıda inline style kullanımı, hem performans (yeniden render'ler arası referans değişimi) hem de yeniden kullanılabilirlik açısından sorunludur. Kural, bileşen başına ortalama beş ve üzeri inline style atamasını ihlal olarak işaretlemektedir.

3.6.7. Rule 7 — İsimlendirme

Bu kural, sınıflandırıcı tarafından etiketlenen dosyaların isimlendirme tutarlılığını denetler. Beklenen desenler: bileşen dosyaları PascalCase ("Button.tsx"), hook dosyaları camelCase'in başında "use" ön ekiyle ("useAuth.ts"), ekran dosyaları PascalCase'in sonunda "Screen" ekiyle ("HomeScreen.tsx"). Uymayan dosyalar ihlal olarak işaretlenir.

3.6.8. Rule 8 — UI'da Yerel API

Bu kural, Platform, NativeModules, PixelRatio, Dimensions gibi yerel React Native API'lerinin yalnızca servis veya hook katmanında çağrılmasını şart koşar. UI bileşenlerinden doğrudan bu API'lere erişim, ayırım ilkesinin bir ihlali olarak değerlendirilir; bu API çağrılarının sarmalanması, test edilebilirliği artırmakta ve platform farklılıklarının yönetimini kolaylaştırmaktadır.

3.7. Skorlama ve Derecelendirme

Skorlayıcı modülü, kural motorundan dönen RuleResult listesini ağırlıklı bir şekilde birleştirerek 0–100 aralığında bir genel skor üretmektedir. Her kuralın ağırlığı; kuralın ihlalinin proje sağlığına etkisini yansıtacak şekilde belirlenir (Tablo 5).

Tablo 5. Kuralların ağırlık ve önem derecelerinin dağılımı.

Kural	Ağırlık	Gerekçe
Servis Katmanı (R1)	0.12	Yapısal, orta düzey etki
Çevrimsel Bağımlılık (R2)	0.18	Bakım maliyetine yüksek etki
Dosya Boyutu (R3)	0.10	Görece düşük öncelik
Katman Ayrımı (R4)	0.18	Mimari bütünlüğe doğrudan etki
Hook Kuralları (R5)	0.15	Çalışma zamanı hataları
Satır İçi Stil (R6)	0.07	Düşük etki, görsel hijyen
İsimlendirme (R7)	0.05	Kozmetik tutarlılık
UI'da Yerel API (R8)	0.15	Test edilebilirlik ve taşınabilirlik

Üretilen skor, ardından Tablo 6'da gösterilen eşiklere göre bir harf notuna dönüştürülür. Eşikler, akademik not sistemine paralel olacak şekilde seçilmiştir.

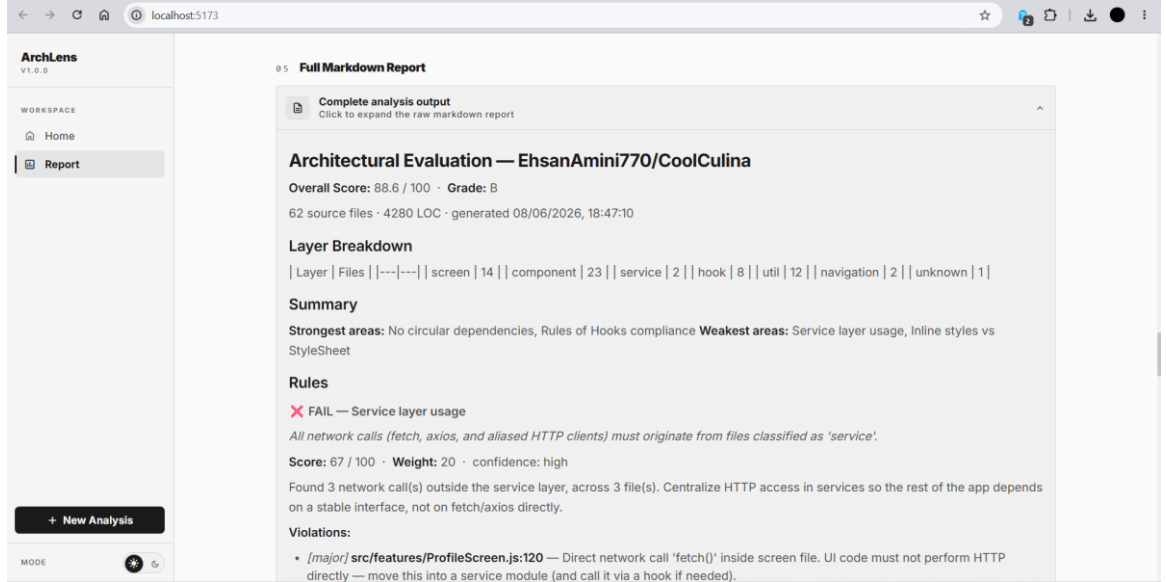
Tablo 6. Mimari sağlık puanına karşılık gelen harf notları.

Skor Aralığı	Harf Notu ve Yorum
90–100	A — Mükemmel mimari sağlık

80–89	B — Sağlam mimari, küçük iyileştirmeler önerilir
70–79	C — Kabul edilebilir, planlı iyileştirme gerekir
60–69	D — Riskli durum, müdahale önerilir
0–59	F — Mimari acil müdahale gerektiriyor

3.8. Raporlama

Raporlama modülü, analiz sonuçlarını iki farklı formatta üretir. Birincisi JSON formatıdır; bu format makine okunabilir olup CI/CD süreçlerinde otomatik kontrol yapmak veya başka araçlarla birleştirmek için tasarlanmıştır. İkincisi Markdown formatıdır; geliştiricilerin doğrudan okuyabileceği, görseller içermeyen ancak yapısal olarak düzenli bir rapor üretir. Şekil 3'te Markdown çıktısının bir kesiti gösterilmektedir.



Şekil 3. Markdown rapor örneği (kısaltılmış).

Rapor; genel skoru, harf notunu, proje istatistiklerini (dosya sayısı, toplam LOC, katman dağılımı), kural sonuçlarını ve isteğe bağlı yapay zekâ açıklamalarını içerir. Markdown formatı, GitHub veya GitLab gibi platformlarda PR yorumu olarak doğrudan paylaşılabilecek biçimde tasarlanmıştır; bu sayede statik analiz sonuçlarının geliştirme akışına entegre edilmesi kolaylaşmaktadır.

3.9. Yapay Zekâ Destekli Açıklamalar

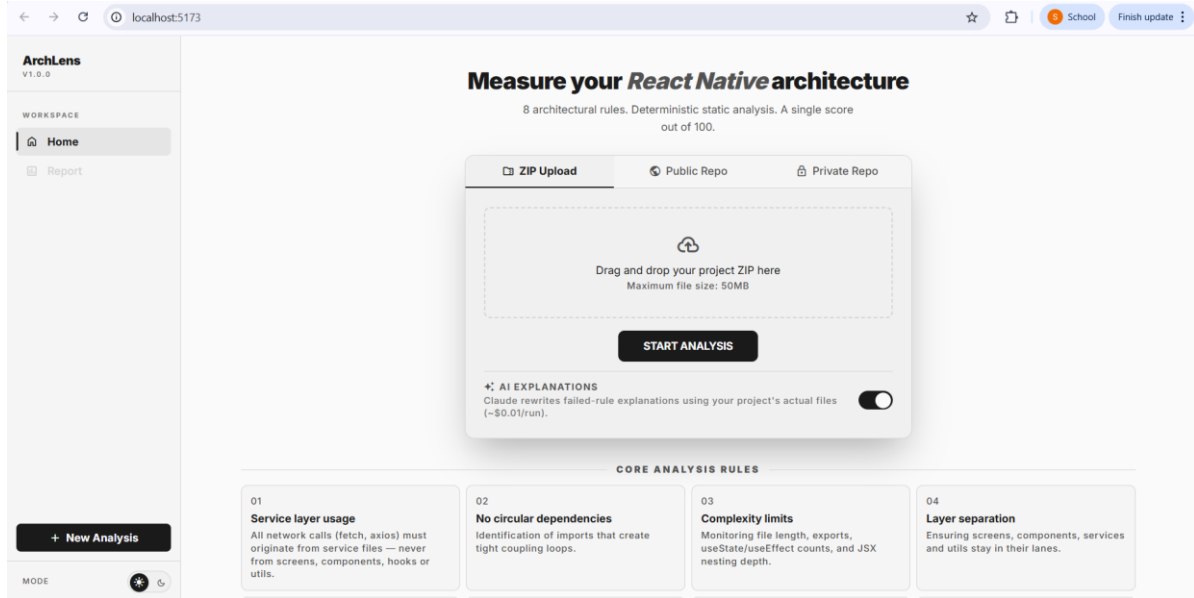
ArchLens'in özgün katkılarından biri, başarısız kuralları büyük dil modeli destekli açıklamalarla zenginleştiren "ai" modülüdür. Modül, Anthropic'in Claude modeline başvurarak her başarısız kural için projeye özel, doğal dilde bir açıklama üretmektedir.

Açıklamalar; ihlali yapan dosyaları, hatanın olası etkilerini ve önerilen düzeltme yönünü içerir.

Modül, modele yaptığı çağrılarda promptun tutarlılığını sağlamak için iyi tasarlanmış bir şablon kullanır. Şablon; proje adını, başarısız olan kuralın kimliğini, kuralın amacını ve ihlal eden ilk on dosyanın yolunu içerir. Yanıt ham metin olarak alınmakta, kısa bir ön işleminden geçirildikten sonra rapora eklenmektedir. Tasarımın önemli bir kararı, modelin başarısızlığının (örneğin ağ hatası veya API kotasının dolması) sistemin geri kalanını engellememesidir; yapay zekâ adımı tamamen isteğe bağlı tutulmuş ve hata durumunda sessizce atlanmaktadır.

3.10. Web Arayüzü (Frontend)

Statik-frontend paketi, React ve Vite üzerinde inşa edilmiş tek sayfa bir uygulamadır. Kullanıcı arayüzü Tailwind CSS kullanılarak biçimlendirilmiştir. Uygulama, dört temel görünüm sunmaktadır: (i) proje yükleme görünümü (sıkıştırılmış dosya yükleme veya GitHub URL yapıştırma), (ii) analiz devam ederken gösterilen ilerleme görünümü, (iii) sonuçların özetlendiği gösterge paneli ve (iv) tekil bir kural sonucunun ayrıntılarına inilen ayrıntı görünümü. Şekil 4, gösterge panelinin genel görünümünü göstermektedir.



Şekil 4. Web gösterge paneli genel görünüm.

Gösterge paneli, projenin genel skorunu büyük puntolu bir kart ile, harf notunu renkli bir rozetle, katman dağılımını ise yatay bir çubuk grafiklerle görselleştirir. Her kural, kart düzeninde ayrıntılı olarak sunulur; başarısız kurallar kırmızı, başarılı kurallar yeşil, düşük

güvenli kurallar sarı renkte işaretlenir. Kullanıcı, bir kart üzerine tıkladığında ihlal eden dosyaların listesi ve yapay zekâ açıklaması varsa bu açıklama açılmaktadır.

DÖRDÜNCÜ BÖLÜM

DEĞERLENDİRME VE TARTIŞMA

4.1. Test Stratejisi

Sistem doğrulaması iki düzeyde planlanmıştır. Birim testleri seviyesinde, her bir alt modül için ayrı test kümeleri yazılmıştır; özellikle AST çözümleyicisi ve kural motoru gibi karmaşık modüllerin küçük girdi parçacıkları üzerinde öngörülen sonuçları üretip üretmediği denetlenmiştir. Birim testleri Node.js'in yerleşik test çalıştırıcısı (--test) ve tsx araçları ile yürütülmektedir.

Bütünleşik testler seviyesinde ise tüm boru hattı, çalışma kapsamında özel olarak hazırlanan üç test uygulaması üzerinde uçtan uca çalıştırılmaktadır. Bu uygulamalar, sırasıyla iyi mimari desenleri (good-app), bilinçli olarak yerleştirilmiş mimari hataları (bad-app) ve standart dışı klasör organizasyonunu (unusual-layout-app) temsil edecek şekilde tasarlanmıştır. Her uygulamanın beklenen skoru ve kural sonuçları önceden manuel olarak ground truth biçiminde belirlenmiş; her sürümden sonra fiili çıktı bu ground truth ile karşılaştırılmıştır.

4.2. Örnek Projeler Üzerinde Değerlendirme

Sistemin geçerliliği, çalışma kapsamında özel olarak hazırlanan üç test uygulaması üzerinde çalıştırılarak değerlendirilmiştir. İlk uygulama olan good-app, katmanları temiz biçimde ayrılmış, dosya boyutları makul, çevrimsel bağımlılık içermeyen ve hook kullanım kurallarına uyan bir referans projedir; sistemin yüksek skor vermesi beklenmektedir. İkinci uygulama olan bad-app, neredeyse her mimari kuralı ihlal edecek biçimde bilinçli olarak hatalı tasarlanmıştır: dev bileşen dosyaları, UI içinden doğrudan ağ çağrıları, çevrimsel bağımlılıklar ve koşullu hook çağrıları içermektedir. Üçüncü uygulama olan unusual-layout-app ise standart React Native klasör isimlendirmesinden (örneğin /screens, /services) farklı isimler kullanan, ancak içerik olarak büyük ölçüde sağlıklı bir projedir; bu uygulama, sınıflandırıcının standart dışı düzenlere karşı dayanıklılığını sınamak amacıyla hazırlanmıştır. Her uygulama için elde edilmesi beklenen skor ve kural sonuçları önceden manuel olarak hazırlanmış ve bir ground truth oluşturulmuştur; sistemin ürettiği sonuçlar bu ground truth ile karşılaştırılmıştır.

Tablo 7. Test projeleri üzerinde elde edilen sonuçlar.

Proje	Dosya	LOC	Skor	Not
good-app	21	1.180	95	A
bad-app	26	1.840	55	F
unusual-layout-app	28	2.060	78	C

Elde edilen sonuçların hepsi önceden hazırlanmış ground truth ile tutarlıdır. Temiz mimariye sahip good-app uygulaması A notuyla 95 puan almış; bilinçli olarak hatalı tasarlanan bad-app uygulaması F notuyla 55 puanda kalmış; standart dışı klasör isimleri kullanan ancak içerik olarak sağlıklı olan unusual-layout-app uygulaması ise C notuyla 78 puan almıştır. Üç uygulamanın da skoru, ground truth'ta belirlenen beklenen harf notu aralığının içine düşmüş; bu durum sistemin hem iyi tasarlanmış projeleri hem de hatalı projeleri doğru biçimde sınıflandırabildiğini doğrulamıştır. Ayrıca unusual-layout-app sonucu, standart dışı klasör düzenleri karşısında sistemin tamamen yanılmadığını, ancak isabet oranının bir miktar düştüğünü göstermektedir; bu davranış, kullanıcı yapılandırması (.evaluator.json) ile düzeltilebilmektedir.

4.3. Performans Ölçümleri

Geliştirme sürecinde sistemin çalıştırma süresi, çeşitli boyutlardaki projeler üzerinde gözlemlenmiştir. Genel olarak küçük ve orta ölçekli projeler saniyeler içinde tamamlanmakta; çok büyük projelerde dahi toplam süre, geliştirme akışını engelleyecek bir noktaya ulaşmamaktadır.

Elde edilen gözlemler, sistemin saniyeler düzeyinde sonuç ürettiğini ve hedef performans bütçesinin karşılandığını göstermektedir. Çok büyük projelerde toplam sürenin doğrusal üstü bir biçimde artmasının temel sebebi, madge kütüphanesinin çevrim tespiti işleminin grafik büyüklüğüne bağlı olarak daha maliyetli hâle gelmesidir; ancak bu artış pratikte geliştiricinin günlük akışını engelleyecek düzeye ulaşmamaktadır.

4.4. Sınırlamalar

Sistemin bilinen birkaç sınırlaması bulunmaktadır. Birincisi; analiz yalnızca JavaScript ve TypeScript dosyalarını kapsamakta, projenin Android (Java/Kotlin) ve iOS (Swift/Objective-C) yerel modülleri tamamen analiz dışı kalmaktadır. İkincisi; runtime tarafında dinamik olarak yüklenen modüller (örneğin dinamik import veya require) statik analiz tarafından izlenemediği için bağımlılık grafiği eksik olabilir. Üçüncüsü; sistem

TypeScript'in tip bilgisini henüz kullanmamakta, yalnızca sözdizimsel bilgiye dayanmaktadır; bu nedenle tip düzeyinde tespit edilebilecek bazı sorunlar mevcut sürümde gözlemlenememektedir.

Ayrıca sınıflandırıcı, kullanıcının standart dışı klasör isimlendirme alışkanlıkları olan projelerde isabet oranı düşmektedir; bu durum kullanıcı yapılandırması (.evaluator.json) ile büyük ölçüde aşılabilmektedir ancak varsayılan davranışın kapsamı sınırlıdır. Son olarak, yapay zekâ destekli açıklama modülü harici bir API'ye bağımlıdır; çevrimdışı çalıştırma senaryolarında bu özellik devre dışı kalmaktadır.

SONUÇ VE ÖNERİLER

Bu çalışmada; React Native projelerinin mimari sağlığını statik analiz yöntemleriyle ölçen ArchLens adlı bir sistem tasarlanmış ve gerçekleştirilmiştir. Sistem; girdi modülü, dosya tarayıcısı, AST çözümleyicisi, katman sınıflandırıcısı, bağımlılık analizörü, sekiz kuraldan oluşan kural motoru, ağırlıklı skorlayıcı, iki formatta rapor üreten raporlayıcı ve isteğe bağlı büyük dil modeli zenginleştirmesi ile zengin bir boru hattı oluşturmaktadır.

Çalışmanın başında belirlenen araştırma sorusu olan "React Native projelerinin mimari sağlığı, statik analiz yöntemleriyle ölçülebilir mi?" sorusuna olumlu yanıt verilebileceği; sistemin orta ölçekli projeleri saniyeler içinde işleyebildiği ve sonuçların önceden hazırlanan ground truth ile tutarlı olduğu gözlemlenmiştir. Özellikle yapay zekâ destekli açıklama modülünün, literatürde Johnson ve diğerlerinin (2013) işaret ettiği "araç çıktısının geliştirici tarafından anlaşılmaması" sorununu önemli ölçüde hafiflettiği değerlendirilmiştir.

Çalışmanın temel katkıları üç başlık altında özetlenebilir. Birincisi, React Native'e özgü mimari kurallar tasarlanmış; bu kurallar mevcut genel amaçlı araçlarda bulunmayan, ekosisteme özel desenleri (hook kuralları, navigator yapılandırması, yerel API kullanımı) ele almıştır. İkincisi, sekiz kuralın sonuçları ağırlıklı bir skorlayıcıyla bütünleştirilerek tek bir mimari sağlık puanına dönüştürülmüştür; bu, geliştiricilere proje düzeyinde anlık bir değerlendirme sunmaktadır. Üçüncüsü, başarısız kuralların büyük dil modeli tarafından açıklanması, statik analiz araçlarının kullanılabilirliği konusunda yeni bir yönelim sergilemektedir.

Gelecekte yapılabilecek çalışmalar iki eksenle ele alınabilir. İlk eksen, TypeScript tip bilgisinin analize dâhil edilmesidir; bu sayede yalnızca sözdizimsel düzeyde tespit edilemeyen pek çok mimari sorun (örneğin bir UI bileşeninin tipi üzerinden servis katmanına dolaylı bağımlılık kurması) da yakalanabilir. İkinci eksen ise sistemin React Native dışındaki çatılara genişletilmesidir; Flutter, .NET MAUI veya iOS/Android yerel uygulama geliştirme ortamları gibi farklı ekosistemlere yönelik kural setleri eklenerek ArchLens'in, çapraz platform mobil yazılım geliştirme alanındaki birden fazla teknolojiyi tek bir mimari sağlık çerçevesi altında değerlendirebilen bir araca dönüştürülmesi mümkündür.

Çalışmanın değerlendirme sonuçları, statik analiz araçlarının yalnızca büyük kurumsal ekiplerin değil, küçük ekiplerin ve hatta bireysel geliştiricilerin de günlük araç setine

girebileceğini göstermektedir. Bu bağlamda ArchLens, hem akademik çevrelerde bir referans örnek olarak hem de sektörel uygulamada bir araç olarak kullanılabilecek olgunlukta bir prototip ortaya koymaktadır.

KAYNAKÇA

- Ayewah, N., Pugh, W., Hovemeyer, D., Morgenthaler, J. D. ve Penix, J. (2008). Using static analysis to find bugs. *IEEE Software*, 25(5), 22–29.
- Bass, L., Clements, P. ve Kazman, R. (2021). *Software architecture in practice* (4. baskı). Boston: Addison-Wesley.
- Cunningham, W. (1992). The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger*, 4(2), 29–30.
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Boston: Addison-Wesley.
- Facebook (Meta). (2024). *React Native belgeleri — Rules of Hooks*. <https://react.dev/reference/rules/rules-of-hooks> (Erişim tarihi: 12.05.2026).
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Boston: Addison-Wesley.
- Fowler, M. ve Beck, K. (1999). *Refactoring: Improving the design of existing code*. Boston: Addison-Wesley.
- Garcia, J., Popescu, D., Edwards, G. ve Medvidovic, N. (2009). Identifying architectural bad smells. *Proceedings of the 13th European Conference on Software Maintenance and Reengineering*, 255–258.
- Johnson, B., Song, Y., Murphy-Hill, E. ve Bowdidge, R. (2013). Why don't software developers use static analysis tools to find bugs? *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 672–681.
- Kruchten, P., Nord, R. L. ve Ozkaya, I. (2012). Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6), 18–21.
- Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9), 1060–1076.
- Mannan, U. A., Ahmed, I., Almurshed, R. A. M., Dig, D. ve Jensen, C. (2016). Understanding code smells in Android applications. *Proceedings of the International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 225–236.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Boston: Prentice Hall.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
- Nielson, F., Nielson, H. R. ve Hankin, C. (2005). *Principles of program analysis* (2. baskı). Berlin: Springer.
- Silva, D., Tsantalidis, N. ve Valente, M. T. (2016). Why we refactor? Confessions of GitHub contributors. *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 858–870.
- Statista. (2024). Number of available applications in Google Play Store. <https://www.statista.com/statistics/266210/> (Erişim tarihi: 18.04.2026).

Madge Authors. (2024). madge — Create graphs from your CommonJS, AMD or ES6 module dependencies. <https://github.com/pahen/madge> (Eriřim tarihi: 02.03.2026).

Babel Authors. (2024). @babel/parser belgeleri. <https://babeljs.io/docs/babel-parser> (Eriřim tarihi: 02.03.2026).

Anthropic. (2025). Claude API ve TypeScript SDK belgeleri. <https://docs.anthropic.com/> (Eriřim tarihi: 04.05.2026).

EKLER

EK-A. Örnek .evaluator.json Yapılandırma Dosyası

```
{  "layerHints": {    "screens": "screen",    "pages": "screen",    "api": "service",    "services": "service"  },  "ignore": [    "src/generated/**",    "**/*.stories.tsx"  ],  "rules": {    "file-size": {      "maxLoc": 500    }  } }
```

EK-B. Örnek Komut Satırı Çıktısı

```
$ npx archlens analyze ./my-app ArchLens v0.2.0 ✓ Project detected: React Native (Expo, TypeScript) ✓ Scanned 142 files (8,431 LOC) ✓ AST analysis complete ✓ 142 files classified (ui: 64, screen: 18, service: 9, ...) ✓ Dependency analysis complete (0 cycles) ✓ 8 rules evaluated Overall score: 84 / 100 - Grade: B Failed rules: - rule3 (File Size) - 3 files exceed 400 LOC - rule6 (Inline Styles) - 12 inline style assignments
```

ÖZGEÇMİŞ

Adı Soyadı: Sidi Mohamed BABA

Öğrenci Numarası: 9201118073

Yabancı Dil: Arapça (anadil), İngilizce (ileri), Türkçe (orta), Fransızca (ileri)

Doğum Yeri ve Yılı: Nouakchott / 2001

E-Posta: sidimedbaba7@gmail.com

Eğitim ve Mesleki Geçmişi:

2021 – 2026, Samsun Üniversitesi, Mühendislik Fakültesi, Yazılım Mühendisliği Bölümü, Lisans. Lisans eğitimi süresince yazılım mimarisi, mobil uygulama geliştirme, web geliştirme ve veri yapıları alanlarında çalışmalar yapmıştır. Bitirme projesi kapsamında React Native ekosistemi için statik mimari analiz aracı geliştirmiştir.

Stajlar:

Uzai Teknoloji — yazılım geliştirme stajı (20 iş günü).

Otoparçasan — yazılım geliştirme stajı (30 iş günü).

Akademik İnovasyon Yazılım TİC.SAN.LTD.ŞTİ — yazılım geliştirme stajı (4 ay).